Energy Based Models for Lean

By William Peng, Holger Molin, Eshaan Barkataki, Matt Chen

The bitter lesson

“The bitter lesson is based on the historical observations that

- Richard Sutton

The bitter lesson

“The bitter lesson is based on the historical observations that

1) AI researchers have often tried to build knowledge into their agents,

- Richard Sutton

The bitter lesson

“The bitter lesson is based on the historical observations that

1) AI researchers have often tried to build knowledge into their agents,

2) this always helps in the short term, and is personally satisfying to the researcher, but

- Richard Sutton

The bitter lesson

“The bitter lesson is based on the historical observations that

1) AI researchers have often tried to build knowledge into their agents,

2) this always helps in the short term, and is personally satisfying to the researcher, but

3) in the long run it plateaus and even inhibits further progress, and

- Richard Sutton

The bitter lesson

“The bitter lesson is based on the historical observations that

1) AI researchers have often tried to build knowledge into their agents,

2) this always helps in the short term, and is personally satisfying to the researcher, but

3) in the long run it plateaus and even inhibits further progress, and

4) breakthrough progress eventually arrives by an opposing approach based on **scaling computation by search and learning**.”

- Richard Sutton

Formal verification as a search problem

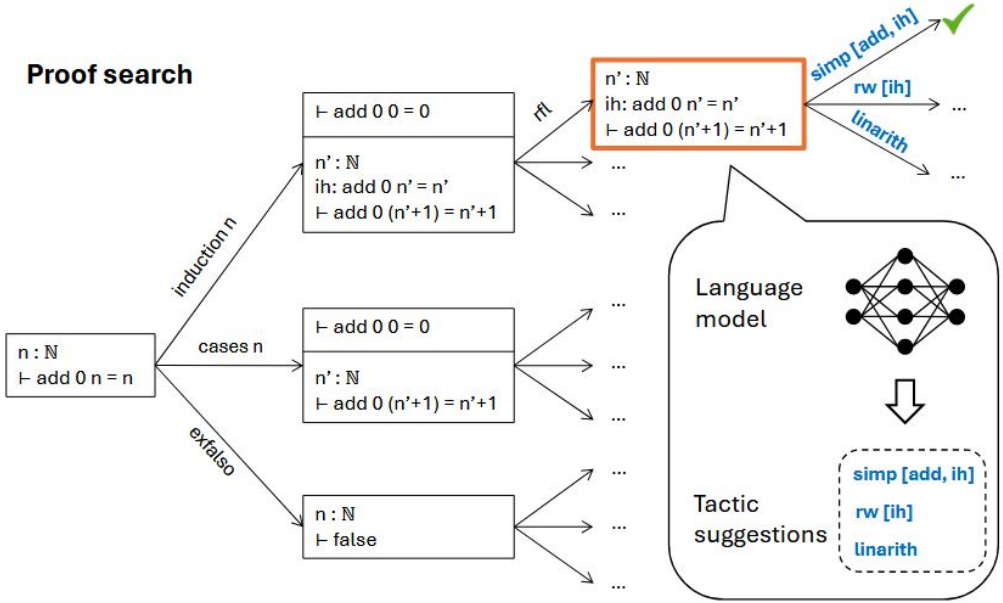


Figure 5. A common architecture for neural theorem proving.

Search problem – continued

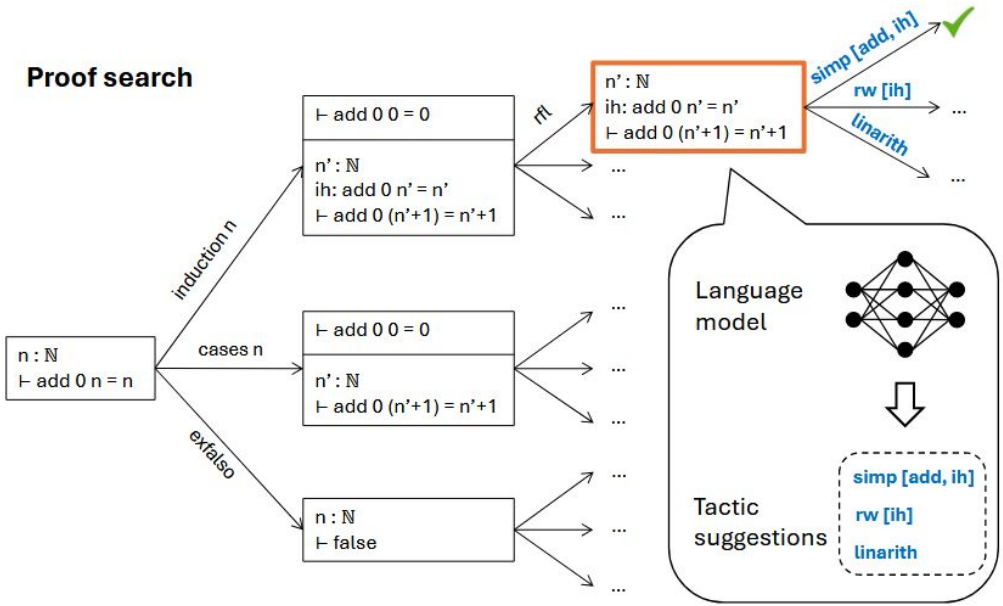


Figure 5. A common architecture for neural theorem proving.

- We can use the energy function as a “verification tool”
- Lower energy → promising candidate
- Higher energy → less promising proof

Current SOTA

Current theorem-proving systems (Aristotle, DeepSeek Prover) rely on two heads:

- **Policy**: an fine-tuned LLM that generates tactics, used in sampling for MCTS
- **Value**: another LLM that outputs the probability of success, used to prune branches

Our idea:

Unify the policy and value into a **single model** (an EBM).

What are EBMs?

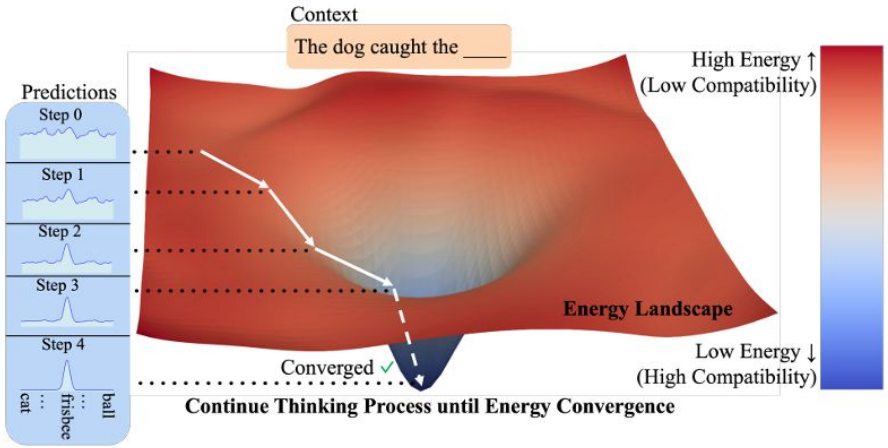
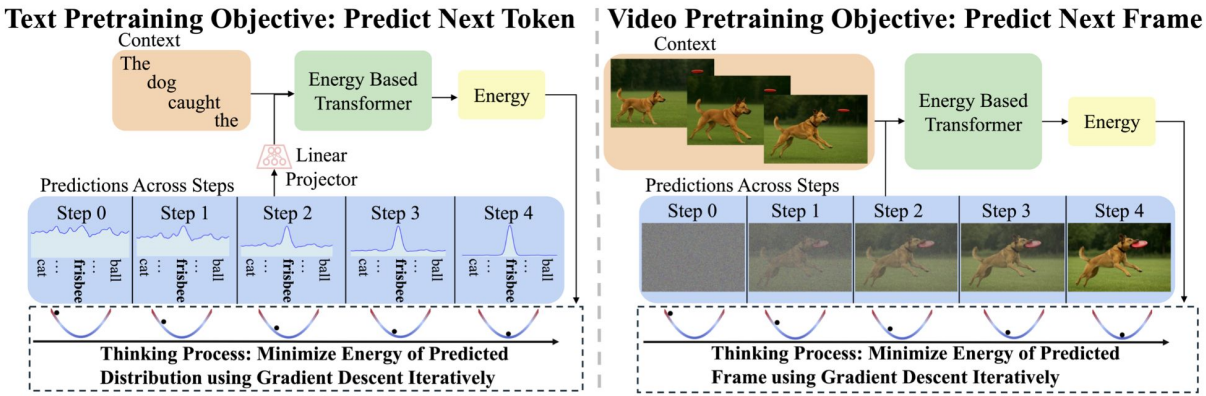


Figure 3: **Thinking Process Visualization.** A learned energy landscape and its optimization through gradient descent, interpreted as a thinking process. In this example, the model predicts a distribution over text tokens, progressively shifting from an initial random distribution toward the target distribution. At each step, the EBM assigns an energy scalar indicating how **compatible** the current prediction is with the context, visualized as the landscape's height (Facet 3). This scalar's convergence allows the model to determine whether the prediction is adequate or if further thinking is necessary. Uncertainty (Facet 2) can be represented by landscapes that are harder to optimize or by landscapes with many local minima, allowing the model to know when it requires more steps to think (Facet 1). Adapted from [57].

How do we predict? Gradient Descent



1. Start with initial noise (every token has an equal probability of being chosen)
2. Get our energy using our model
3. Calculate the gradient of our *energy* (we want to lower this) w.r.t *our input* (initial noise)
4. Change our initial noise using the gradients

EBMs Versus other Generative Models

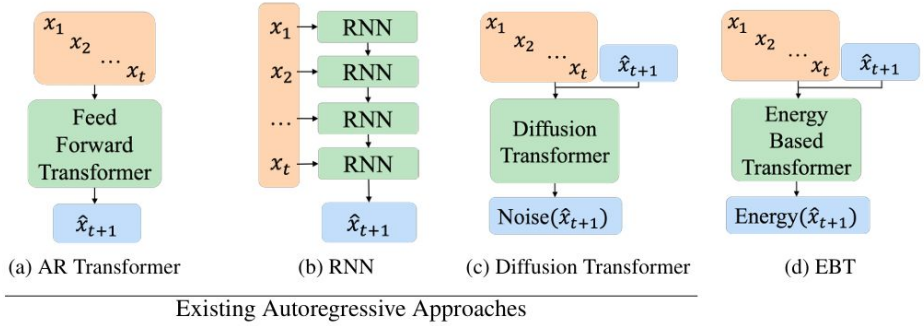
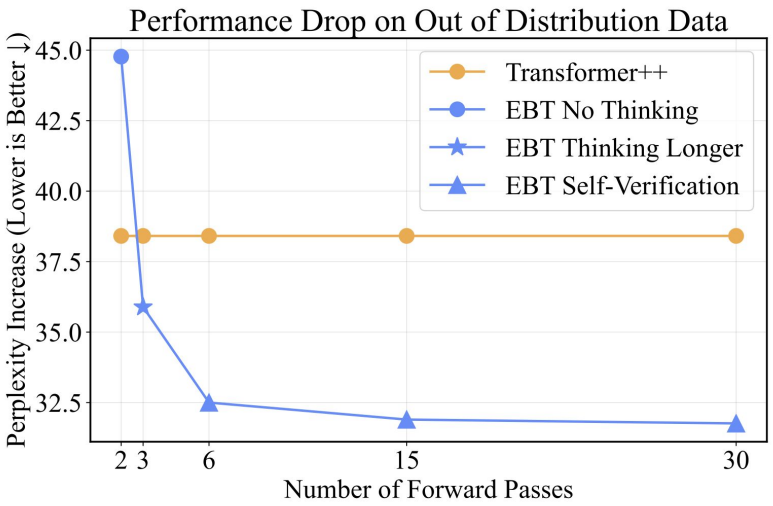


Figure 1: **Autoregressive Architecture Comparison.** (a) Autoregressive (AR) Transformer is the most common, with (b) RNNs becoming more popular recently [22, 23]. (c) Diffusion Transformers [26] (which are often bidirectional but can also be autoregressive) are the most similar to EBT, being able to dynamically allocate computation during inference, but predict the noise rather than the energy [27, 28]. Consequently, diffusion models cannot give unnormalized likelihood estimates at each step of the thinking process, and are not trained as explicit verifiers, unlike EBTs.

1. Transformers/RNNs: seq → distribution over tokens
2. Diffusion: seq + noised distribution→ denoised distribution
3. EBT: seq + potential distribution→ energy scalar
   a. This scalar measures how good the potential output.

Benefits of Energy Based Models vs. Alternatives

1. Model Uncertainty
   a. Higher energy → more uncertain about the answer
2. Dynamic Allocation of Computation
   a. As humans, for a hard problem → think longer. Easier problem → think shorter.
   b. EBMs mimic this: more forward passes → better generalization
3. Trained on Verifying Predictions
   a. *Verifying Predictions* is easier to train than *Generating Predictions*

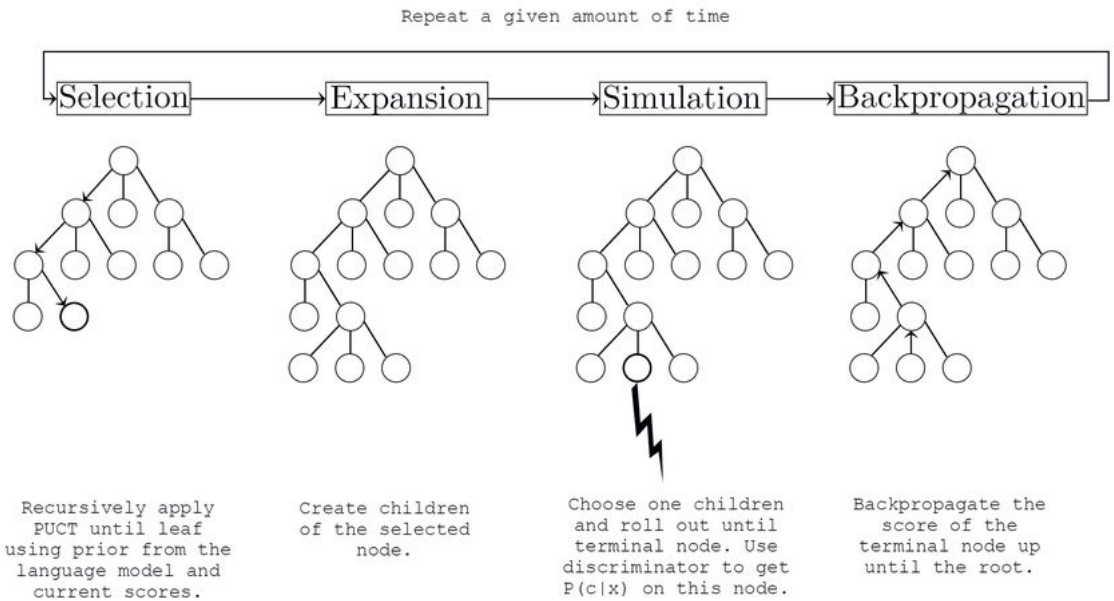


(a) OOD Thinking Performance Comparison.

How to solve math as search problem with EMB

Use EBM + Langevian to sample chunks of tactics

Use energy score as the value to guide MCTS searches



Implications and Risks:

Implications

1. Training needs to be predicting next chunk
2. How to separate training data into chunks
3. Need a continuous representation of tactics

Risks

1. Local minima problem
2. “Cliff” problem

The bitter lesson, continued

One thing that should be learned from the bitter lesson is the great power of general purpose methods, of methods that continue to scale with increased computation even as the available computation becomes very great. The two methods that seem to scale arbitrarily in this way are *search* and *learning*.

The second general point to be learned from the bitter lesson is that the actual contents of minds are tremendously, irredeemably complex; we should stop trying to find simple ways to think about the contents of minds, such as simple ways to think about space, objects, multiple agents, or symmetries. All these are part of the arbitrary, intrinsically-complex, outside world. They are not what should be built in, as their complexity is endless; instead we should build in only the meta-methods that can find and capture this arbitrary complexity. Essential to these methods is that they can find good approximations, but the search for them should be by our methods, not by us. We want AI agents that can discover like we can, not which contain what we have discovered. Building in our discoveries only makes it harder to see how the discovering process can be done.

Methods

We’ll take an existing pretrained, math-finetuned model (e.g. DeepSeek Math-7B), finetune a projection head, and implement inference logic.

Open Questions

Fine-tuning vs. training from scratch

- Fine-tuning: embeddings?
- Agentic system (like Aristotle)?

Next steps

- Make a CS id: CSID | Stanford Computer Science
- Divide pipeline work- benchmarks, training, architecture, etc.
- Why do we believe in this?
- How can we test that?
- How can we convince a skeptic that this is true?
- How do we train the ebm in a stable way?
  - Contrastive - negative samples?
  - What does the structure of Lean change about model training
  - Data generation?
- Benchmarks: miniF2F, Veribench
- Lean training data
- Interfacing with Lean (v4.26): stanford-centaur/PyPantograph: A Machine-to-Machine Interaction System for Lean 4.
- brando90/snap-cluster-setup

- Code Verification
- Scalable Oversight of Code
  - tests/theorems → automate writing theorems
- Data quality
  - train (post/pre/cont) train a model for all Lean
  - doremi, dodge
    - agentic data
- Replacing pre-training with “all training”
- EBM